

Sistema Integral de Gestión de Eventos

Arquitectura de Software · SRS, SAD y prototipo funcional

Daniel Cristancho Samuel Emperador Sebastián Sánchez Diego Coronado

Septiembre de 2026

Pontificia Universidad Javeriana · HEXACORE

Qué vamos a ver

1. Especificación de requisitos (SRS)
2. Modelo de dominio
3. Stakeholders, ASR y restricciones
4. Vistas de la arquitectura
5. Decisiones y riesgos
6. Bitácora arquitectónica
7. Prototipo funcional
8. Atributos de calidad
9. Cierre

Especificación de requisitos (SRS)

El sistema en una frase

Alcance

Gestiona el ciclo completo de un evento masivo —conciertos, partidos, ferias— **desde la venta de entradas hasta la operación en campo** el día del evento.

Seis frentes funcionales

- Boletería y validación de QR
- Mercado secundario (reventa)
- Personal, turnos y logística
- Alimentos y bebidas
- Parqueaderos
- Administración y emergencias

Queda fuera

- Procesamiento de pagos (*pasarela externa*)
- Envío físico de notificaciones (*proveedor externo*)
- Contabilidad de la organizadora
- Integración con venta de terceros

El SRS dice **qué** debe hacer el sistema; el SAD, **cómo**. Por eso en el SRS no se nombra ningún motor de base de datos, lenguaje ni biblioteca: un requisito bien escrito sobrevive a un cambio de tecnología.

Actores y las tres interfaces

Actor	Qué hace
Cliente	Compra, revende, pide comida, reserva parqueadero
Empleado	Turnos, asistencia, valida QR, reporta incidentes
Organizador	Crea eventos, asigna personal, consulta reportes
Administrador	Cuentas, roles, recintos, conciliación
Supervisor	Activa y coordina la evacuación
<i>Pasarela de pagos</i>	<i>Externo: cobros y reembolsos</i>
<i>Notificaciones</i>	<i>Externo: correo, push, SMS</i>

Interfaces

- **Portal web de clientes** — cartelera, compra, reventa, cuenta.
- **Portal web administrativo** — eventos, personal, recintos, reportes.
- **App móvil (Flutter)** — la única que se usa *dentro* del evento; sirve a Cliente y a Personal, y debe tolerar conectividad intermitente.

Un mismo usuario puede tener varios roles: quien trabaja en un evento también compra entradas para otro.

32 casos de uso en seis bloques

Bloque	Casos de uso	Alcance
Boletería	CU-001 – 005	Consulta, compra, validación de QR, cancelaciones, promociones
Mercado secundario y personal	CU-006 – 010	Reventa segura (complejo) , personal, turnos, asistencia, evacuación (complejo)
Alimentos y bebidas	CU-011 – 015	Pedidos, menú e inventario, preparación, entrega por QR
Logística y operación	CU-016 – 020	Planificación, personal operativo, monitoreo, incidentes
Parqueaderos	CU-021 – 025	Reserva, ingreso/salida, espacios, cobro, ocupación
Administración general	CU-026 – 032	Eventos, cuentas, roles, recintos, proveedores, pagos, reportes

Los dos casos complejos son el hilo conductor del SAD

CU-006 (Mercado Secundario) por la concurrencia sobre un recurso único, y **CU-010 (Evacuación)** por la entrega garantizada de instrucciones bajo presión de tiempo.

Requisitos no funcionales con umbral verificable

RNF	Enunciado	Atributo
RNF-01	Ninguna entrada publicada puede venderse a dos compradores	Consistencia
RNF-02	Al transferirse, el QR anterior queda invalidado antes de emitir el nuevo	Consistencia
RNF-03	Sigue atendiendo ante la caída de una instancia de cualquier servicio	Disponibilidad
RNF-04	Ante una caída, se recupera sin intervención manual	Disponibilidad
RNF-05	Nunca almacena datos completos de tarjetas de pago	Seguridad
RNF-06	Toda petición pasa autenticada y autorizada por rol desde el Gateway	Seguridad
RNF-07	La consulta de disponibilidad responde con baja latencia en la apertura	Desempeño
RNF-10	El servicio de Entradas escala sin degradar los demás dominios	Escalabilidad
RNF-11	Toda operación financiera queda auditada	Trazabilidad
RNF-15	Una versión nueva se despliega sin interrumpir el servicio	Desplegabilidad
RNF-17	Al activarse una evacuación, todo el personal recibe su instrucción	Seguridad física
RNF-18	Cada microservicio puede probarse aislado, sin levantar los demás	Comprobabilidad

Son 18 en total. Cada uno fija una **métrica y un umbral**, no una intención: es lo que permite que la sección de atributos de calidad de esta presentación muestre números y no adjetivos.

Modelo de dominio

Conceptos derivados de los 32 casos de uso

Concepto	Qué es
Evento	Actividad con fecha, recinto, localidades y aforo
Entrada	Boleto con QR único, ligado a su propietario actual
Publicación de reventa	Oferta de una entrada ya adquirida
Recinto / Zona	Lugar físico y sus subdivisiones con aforo
Usuario / Cuenta	Persona registrada con sus roles
Rol / Permiso	Acciones autorizadas sobre un módulo
Personal	Trabajador con rol operativo
Turno	Periodo de trabajo con horario y zona

Concepto	Qué es
Registro de asistencia	Marca de ingreso y salida en un turno
Parqueadero / Espacio	Zona de estacionamiento y cada cupo
Pedido	Solicitud de alimentos durante el evento
Establecimiento	Punto de venta y su menú
Pago	Transacción procesada por la pasarela externa
Proveedor	Tercero contratado para la operación
Incidente	Anomalía, hasta activar una evacuación
Reporte	Consulta agregada para decidir

La decisión que atraviesa todo el modelo: la **Entrada** tiene un solo propietario en cada momento y un QR que lo identifica. Toda la complejidad del CU-006 nace de ahí —transferir la propiedad e invalidar el QR anterior tienen que ocurrir juntos o no ocurrir.

Stakeholders, ASR y restricciones

Stakeholders e intereses

Stakeholder	Interés / expectativa
Cliente	Comprar y revender de forma confiable, y recibir su QR sin fallas en el ingreso
Organizador	Crear eventos, asignar personal y proveedores, monitorear en tiempo real
Empleado / Personal	Consultar turnos, pedir cambios y registrar asistencia sin fricción
Administrador	Gestionar usuarios, roles y recintos con control de acceso auditable
Supervisor de emergencia	Activar y coordinar evacuaciones con información confiable y actual
Proveedor externo	Ser asignado correctamente y recibir la información logística necesaria
Equipo Hexacore	Sustentar una arquitectura que satisfaga los atributos priorizados
Profesor / Evaluador	Verificar que el SAD cumpla la plantilla y que las decisiones estén justificadas

Árbol de utilidad — los escenarios que concentran el riesgo

ASR	Escenario	Atributo	CU	Import.	Dific.
ASR-01	Dos compradores van por la misma entrada a la vez; solo uno completa	Consistencia	CU-006	Alta	Alta
ASR-02	Cae una instancia durante el ingreso masivo; el balanceador redirige	Disponibilidad	CU-002	Alta	Media
ASR-03	Acceso no autorizado a datos de pago, rechazado y auditado	Seguridad	CU-001	Alta	Media
ASR-04	Miles de usuarios consultan y compran en la apertura de venta	Rendimiento	CU-001/5	Alta	Alta
ASR-06	Sube el tráfico de Entradas en preventa; escala sin afectar a otros	Escalabilidad	CU-006	Media	Media
ASR-12	Corregir Pedidos con el evento en curso, sin detener la venta	Desplegabilidad	Todos	Media	Media
ASR-13	El personal recibe su instrucción de evacuación en segundos	Seguridad física	CU-010	Alta	Alta

Los tres (Alta, Alta) se respaldan con medición, no con razonamiento

ASR-01 y ASR-04 se demuestran hoy con pruebas ejecutables; ASR-13 depende del CU-010, todavía no implementado. Los otros cuatro —ASR-05, ASR-09, ASR-10 y ASR-14— están en el SAD con el mismo detalle.

Técnicas

- Microservicios con **base de datos independiente por dominio** (ADR-01). Cualquier solución debe respetar esa separación.
- Soportar picos de **miles de usuarios concurrentes** en la apertura de venta y el ingreso masivo.

Organizacionales

- Equipo de **4 integrantes**, 8 casos de uso por persona. Condiciona qué prototipa cada uno.

Convencionales

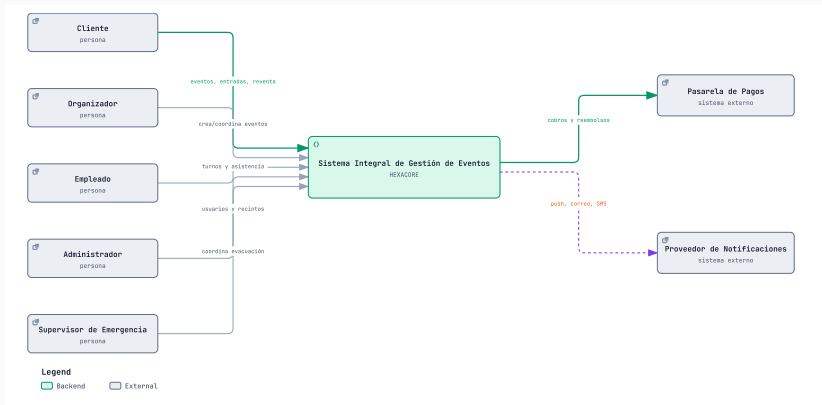
- Documento basado en el estándar **arc42**; no pueden omitirse secciones.
- Fechas y formato de entrega definidos por el curso.

Consecuencia práctica

Con 4 personas y 32 casos de uso, la pregunta no es si se implementa todo, sino **qué se implementa de punta a punta** y con qué evidencia.

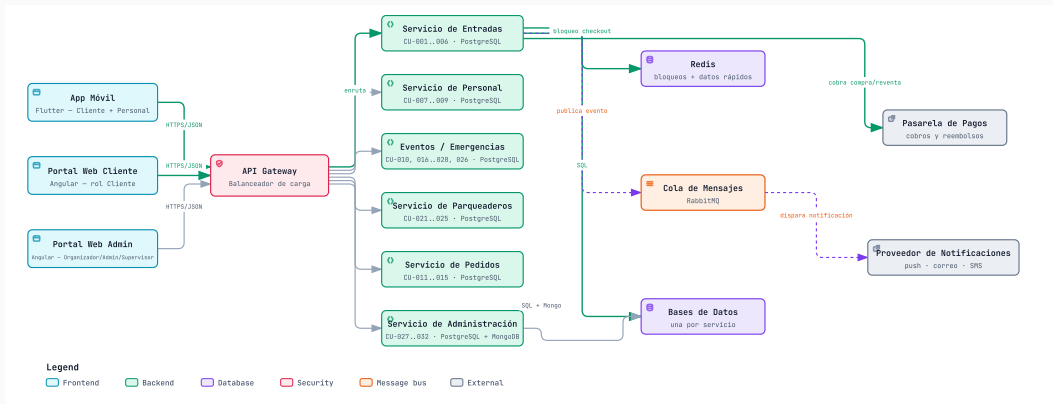
Vistas de la arquitectura

Contexto — C4 nivel 1



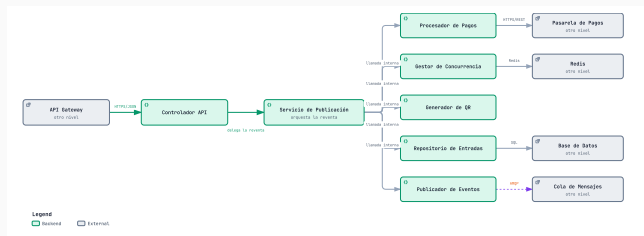
Cinco roles de usuario y **dos sistemas externos**: la pasarela de pagos y el proveedor de notificaciones. Ambos están fuera del alcance a propósito, y el sistema debe seguir siendo correcto cuando no respondan.

Contenedores — C4 nivel 2



Tres interfaces sobre un único API Gateway (ADR-02), que centraliza autenticación, autorización y enrutamiento. Detrás, microservicios por dominio con su propia base de datos, más Redis (bloqueo distribuido y datos de baja latencia) y RabbitMQ (operaciones asíncronas).

Componentes — Servicio de Entradas y Mercado Secundario

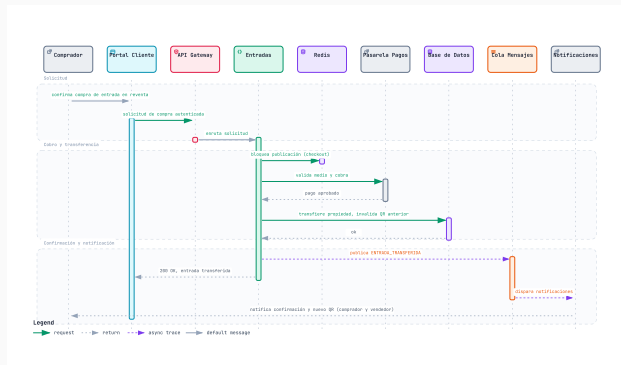


El flujo de una reventa

1. El **Controlador API** recibe lo que enrutó el Gateway.
2. El **Servicio de Publicación** orquesta la operación.
3. El **Gestor de Concurrencia** bloquea la entrada en Redis.
4. El **Procesador de Pagos** cobra contra la pasarela.
5. El **Generador de QR** invalida el código anterior y emite el nuevo.
6. El **Repositorio** persiste el cambio de propiedad.
7. El **Publisher de Eventos** avisa por la cola.

Los otros tres servicios de dominio (Personal, Eventos/Emergencias, Parqueros) se documentan al mismo nivel en el SAD.

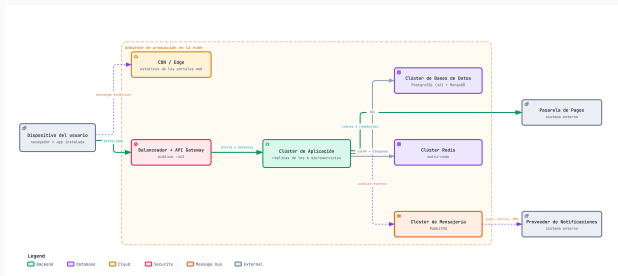
Procesos — reventa de una entrada (CU-006)



1. El comprador confirma la compra.
2. El Gateway enruta la solicitud **ya autenticada**.
3. Entradas **bloquea la publicación en Redis**: nadie más puede adquirirla.
4. Se cobra por la pasarela de pagos.
5. Se transfiere la propiedad y **se invalida el QR anterior**, en una sola transacción.
6. Se publica ENTRADA_TRANSFERIDA en la cola.
7. Notificaciones envía el QR nuevo, desacoplado del flujo principal.

Evacuación (CU-010) sigue el mismo patrón asíncrono: identificar zonas, confirmar protocolo, difundir por la cola y monitorear hasta el cierre.

Vista física — producción, y lo que corre hoy



Producción — CDN para estáticos, balanceador + Gateway, clúster de aplicación con réplicas por microservicio, y clústeres de base de datos, Redis y mensajería.

Desarrollo y pruebas — misma topología a menor escala: una réplica por servicio, sin CDN, y Redis, cola y bases en un único nodo compartido.

Lo que se demuestra en vivo

El sistema repartido en dos computadores: la capa de datos en uno (`--rol datos`) y los servicios más el Gateway en otro (`--rol servicios --datos <IP>`).

Modelo de datos — una base por microservicio

Entradas / Mercado Secundario

- Entrada(id, evento, localidad, **propietario**, codigoQR, estado)
- PublicacionReventa(id, entrada, vendedor, precio, estado)
- TransaccionReventa, HistorialPropietario, Compra, Ingreso, Cancelacion

Eventos / Emergencias

- Evento, Recinto, Zona, Incidente
- Turno, RegistroAsistencia

Pedidos

- Pedido, Establecimiento, Producto, ReservaInventario, TransaccionPedido

Administración

- Usuario, Rol, UsuarioRol
- Sesion, TokenCuenta, AuditoriaCuentas

La regla que lo sostiene

Las referencias entre dominios se resuelven **en la aplicación**, con el identificador, y **no con llaves foráneas físicas** entre bases distintas. Es lo que permite que cada servicio evolucione y se despliegue solo (ADR-01).

PostgreSQL para lo transaccional; MongoDB queda reservado para Reportes (ADR-06).

Decisiones y riesgos

Once decisiones arquitectónicas registradas

ADR	Decisión	Por qué, y qué se descartó
ADR-01	Microservicios por dominio	Escalar Entradas sin sobre-aprovisionar el resto; se descartó el monolito modular
ADR-02	API Gateway + balanceador	Un solo punto seguro de entrada; evita duplicar seguridad en tres interfaces
ADR-03	Redis para bloqueo y caché	Evita la doble venta sin volver la base el cuello de botella
ADR-04	Cola de mensajes asíncrona	Un fallo del proveedor externo no puede tumbar la compra
ADR-05	Angular en los portales	Módulos, guards e interceptors; trabajo en paralelo
ADR-06	PostgreSQL + MongoDB	ACID para lo transaccional; esquema flexible para reportes
ADR-07	Una sola app móvil para dos roles	El rol se resuelve en el login; menos configuración duplicada
ADR-08	Migración de la app a Flutter	ADR-05 omitió que la app debe llegar a Android e iOS
ADR-09	NestJS en el backend	Decidido con PoC medida, no por preferencia
ADR-10	RabbitMQ como cola	Decidido con PoC medida, no por popularidad
ADR-11	RS256 y dos tokens	Solo Administración firma; los demás solo verifican

Dos decisiones tomadas con evidencia, no con opinión

ADR-09 — Lenguaje del backend

Mismo endpoint en ambos, 100 conexiones, cero errores:

	req/s	p97,5	RSS
NestJS	21.071	5 ms	156 MB
Spring Boot	91.441	1 ms	335 MB

Spring Boot rinde 4,3× más, y conviene decirlo sin matizarlo. Pero el umbral de RNF-07 es $p95 \leq 500$ ms y el peor caso medido fue **7 ms**: setenta veces por debajo. **El rendimiento no decide.**

Deciden Modificabilidad y Comprobabilidad —NestJS impone el mismo modelo de módulos que el equipo ya usa en Angular— y el cronograma: 4 personas, 6 servicios, un solo lenguaje entre frontend y backend.

ADR-10 — Motor de la cola

Evento real ENTRADA_TRANSFERIDA, 500 mensajes:

Modo duradero	p50	p95
RabbitMQ	162 ms	282 ms
Kafka	31,6 ms	57,9 ms

Con durabilidad Kafka es cinco veces mejor. Pero el umbral de RNF-17 es 10 s y el peor caso fue 282 ms: treinta y cinco veces por debajo. Otra vez, la latencia no decide.

Decide el uso: esto es una **cola de tareas**, no un registro de eventos con reproducción histórica. RabbitMQ trae **DLQ y reintentos nativos** —que ASR-07 y ASR-13 exigen— con la mitad de memoria y de configuración.

Riesgos técnicos y su mitigación

Riesgo	Prioridad	Mitigación	Contingencia
Concurrencia entre compradores en el mercado secundario	Alta	Bloqueo distribuido con TTL en Redis antes de pagar	Rechazar la segunda y notificar de inmediato
Caída del API Gateway como punto único	Alta	Varias instancias, balanceador y <i>health checks</i>	Failover hacia instancia o región de respaldo
Indisponibilidad de la pasarela de pagos	Alta	Timeouts, reintentos y cola de reconciliación	Marcar pendiente y permitir reintento manual
Picos en apertura de venta o ingreso masivo	Alta	Autoescalado de Entradas y caché en Redis	Sala de espera virtual que limita la concurrencia
Fallo de notificaciones en una emergencia	Media	Reintentos y canal secundario (SMS si falla push)	Escalar el aviso mediante el personal en sitio
Pérdida de conexión móvil durante una evacuación	Media	Caché local de instrucciones y reconexión	El personal de seguridad releva la comunicación

Bitácora arquitectónica

Cómo se registró el camino, no solo el resultado

Bitácora grupal

- `Documentation/Work/BitacoraArquitectonica.md`
- Registra **alternativas consideradas y descartadas**, no solo lo elegido: es de donde salieron los ADR.
- Incluye las dos pruebas de concepto medidas (PoC-04 lenguaje, PoC-05 cola) con sus datos crudos en `App/PoCs/`.
- Documenta las decisiones **revertidas**: ADR-05 quedó parcialmente superado por ADR-08 al detectar que la app debía llegar también a iOS.

Bitácora individual

- Cada integrante lleva la suya sobre los casos de uso que le tocaron.
- El historial de Git la respalda: ramas `feature/<nombre>` y **pull request con revisión** obligatorio hacia `develop`.

Trazabilidad viva

Los mensajes de commit explican **el problema, la solución y cómo se verificó**, de modo que la bitácora no es un documento aparte que se desincroniza sino el propio historial del repositorio.

Prototipo funcional

Qué significa aquí “implementado”

Un caso de uso cuenta como completo solo si es end-to-end

GUI ➡ backend ➡ base de datos ➡ servicios de apoyo (Redis, RabbitMQ, pasarela de pagos), y con pruebas de integración que lo ejerciten por HTTP contra el sistema real.

Lo que está desplegado y corriendo

- 4 microservicios NestJS con base propia
- API Gateway Nginx con auth_request
- 2 portales Angular + app móvil Flutter
- PostgreSQL, Redis, RabbitMQ
- Pasarela de pagos y correo simulados

La dependencia transversal, resuelta

- **CU-027 (Autenticación)** está completo: registro, verificación por correo, login, renovación, recuperación y cierre de sesión.
- Es la que habilita a todas las demás: sin ella, ningún CU de Cliente o de Personal puede demostrarse.

Estado por integrante — casos de uso complejos

Integrante	CU	Caso de uso	GUI	API	BD	Pruebas
Samuel Emperador	CU-006	Mercado secundario (<i>complejo</i>)	✓	✓	✓	✓
	CU-026	Gestión de eventos (CRUD)	●	✓	✓	✓
	CU-007–010	Personal y evacuación	✗	✗	✗	✗
	CU-024–025	Parqueadero: cobro y ocupación	●	✗	✗	✗
Daniel Cristancho	CU-001–005	Boletería completa	✓	✓	✓	✓
	CU-021–023	Parqueadero: reservas y accesos	●	✗	✗	✗
Sebastián Sánchez	CU-027	Cuentas y autenticación	✓	✓	✓	✓
	CU-011–012	Pedidos y menú	✓	✓	✓	✓
	CU-013–015	Preparación y entrega	●	●	✓	✗
	CU-028–029	Roles y recintos	✗	✗	✗	✗
Diego Coronado	CU-017–018	Personal operativo, turnos y asistencia	✓	✓	✓	✓
	CU-019	Monitoreo del evento	●	●	✓	●
	CU-016, 020	Logística e incidentes	✗	✗	✗	✗
	CU-030–032	Proveedores, pagos y reportes	✗	✗	✗	✗

✓ completo ● parcial ✗ no implementado

CU-006 — Mercado secundario, el caso complejo end-to-end

Lo que ejercita una sola compra

- **GUI:** portal web (mercado y “mis entradas”) y app móvil
- **Gateway:** valida el token antes de enrutar
- **Redis:** bloqueo del checkout con dueño y TTL
- **PostgreSQL:** transferencia de propiedad, QR nuevo e historial, en **una sola transacción**
- **Pasarela:** cobro con clave de idempotencia
- **RabbitMQ:** ENTRADA_TRANSFERIDA hacia notificaciones y liquidación

Caminos alternos cubiertos por pruebas

CU-006A cambio de precio · CU-006B retiro · CU-006D expiración · CU-006E entrada no revendible · CU-006F evento sin reventa · CU-006G pago rechazado · CU-006H compra simultánea · CU-006I pasarela caída

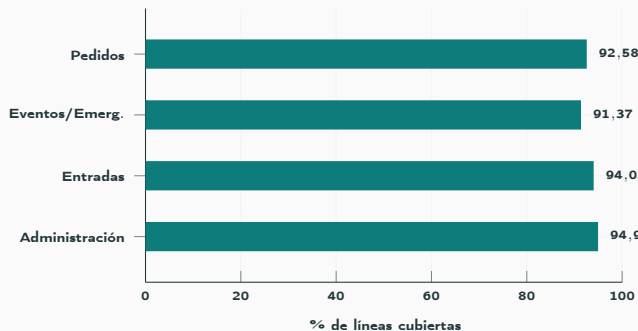
Post-condiciones verificadas contra la base

- La propiedad cambió al comprador
- El QR anterior **no existe en ninguna fila** (RNF-02)
- El historial queda EMISION → REVENTA (RNF-11)
- El reparto cuadra al céntimo: $\text{precio} = \text{comisión} + \text{neto}$

Lo que el historial no admite

Un disparador rechaza UPDATE y DELETE sobre `historial_propietarios`: la cadena de propiedad no se puede reescribir desde la aplicación.

Cobertura de pruebas de integración sobre el backend



20 suites de integración que hablan por HTTP contra el sistema real, más 59 pruebas e2e en Eventos/Emergencias.

Por qué no es 100 %, dicho de frente

Lo que falta son tres cosas distintas:

- Rutas de fallo de infraestructura (Redis caído, canal de RabbitMQ cerrado) y validaciones que solo se disparan al arrancar mal. Llegar ahí pide **inyección de fallos**.
- Un **artefacto de medición** en los DTO: el plugin de Swagger genera en cada clase un método cuyo mapa de fuentes abarca todo el bloque de propiedades. Se ve a simple vista: hay **comentarios** marcados como no cubiertos.

No se excluyen los DTO para maquillar el porcentaje: esconderlos escondería también sus funciones de mapeo, que sí se prueban.

Atributos de calidad

Cinco atributos, medidos con una sola orden

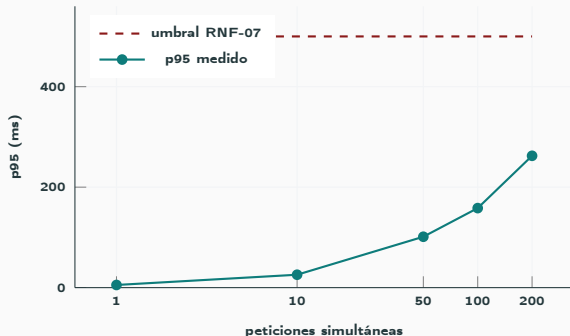
```
cd App/infra && ./atributos-calidad.py
```

Corre las cuatro suites **contra el sistema desplegado** —a través del Gateway, por donde entra el tráfico real— y deja el informe escrito en `informe-atributos/INFORME.md`.

Atributo	RNF	Resultado	Referencia
Desempeño	RNF-07, 08	p95 189,9 ms con 200 simultáneas; 0 fallidas	umbral 500 ms
Disponibilidad	RNF-03, 04	En pie 1,3 s tras kill -9	exigido ≤ 30 s
Seguridad	RNF-05, 06	11 de 11 rutas responden 401 sin token	exigido 0% abiertas
Consistencia	RNF-01, 02	1 checkout y 49 rechazos con 50 compradores	exigido exactamente 1
Desplegabilidad	RNF-15	88 s de cero a sistema utilizable	un solo guion

Si una suite falla, su fila queda marcada como fallida y el guion termina con error: **no rellena el hueco con la cifra de la corrida anterior**.

Desempeño — RNF-07 bajo carga creciente



Simult.	p95	pet/s
1	5,3 ms	253
10	25,7 ms	630
50	101,3 ms	693
100	158,3 ms	735
200	262,4 ms	781

Peor caso medido **262,4 ms** frente a un umbral de **500 ms**, con **0** peticiones fallidas en toda la prueba.

Las tácticas detrás: caché con Cache-Control en la cartelera, réplicas tras el balanceador, resolución del destino **en cada petición**, y pool de conexiones acotado por servicio.

Consistencia — RNF-01: cero ventas duplicadas

Escenario	Resultado
50 compradores a la vez sobre la misma entrada	1 checkout, 49 rechazados
Pagar con la reserva ya caducada	1 cobro aprobado
Los dos pagan simultáneamente	1 pago, 1 QR
Dos puertas escanean el mismo QR	1 ingreso; el aforo sube 1

Ocho tácticas lo sostienen: bloqueo pesimista, escrituras condicionales que miran cuántas filas cambiaron, bloqueo distribuido con dueño, claves de idempotencia, invariantes en la propia base, una sola transacción por transferencia, historial inmutable y el orden anular-antes-de-reembolsar.

La red que queda si la lógica fallara

Invariantes que la base de datos no admitiría violar:

- Índice único parcial: **una sola publicación activa** por entrada
- `UNIQUE` sobre `ingresos.entrada_id`: un QR no ingresa dos veces
- `CHECK`: `precio = comisión + neto`
- `CHECK`: una transacción aprobada **exige** referencia de la pasarela

Por qué importa el orden

En la cancelación se **anula primero** y se **reembolsa después**: así la entrada no se puede usar en la puerta mientras el dinero está en vuelo. Si la pasarela rechaza, las entradas vuelven a ser válidas.

Seguridad y disponibilidad — RNF-03 a RNF-06

Seguridad (RNF-06)

Comprobación	Resultado
Rutas de negocio sin token	11 de 11 dan 401
Formas de token inválido	8 rechazadas
Revocación al cerrar sesión	inmediata
Si Redis no responde	503 , no se entra

Se rechazan token firmado con otra clave, alg: none, HS256 usando la clave pública como secreto, caducado, de otro emisor y manipulado.

RNF-05: de los pagos solo se guarda una **referencia opaca**. Ni el esquema ni los eventos publicados tienen campo para número de tarjeta.

Disponibilidad (RNF-03, RNF-04)

Dependencia caída	El login
Redis	200 en 90 ms
RabbitMQ	200 en 95 ms
PostgreSQL	503 honesto
kill -9 del proceso	en pie en 1,3 s

Fallar en claro también es disponibilidad

Sin base de datos **no hay login posible**. Lo que se exige entonces no es un 200 falso, sino un **503 rápido** que un balanceador entienda como “prueba otra instancia” —y que /salud lo diga.

Desplegabilidad — dos computadores, un solo guion

Un único guion levanta el sistema completo

```
cd App/infra && ./iniciar.sh
```

De cero a sistema **utilizable** en **88 s**: levanta la infraestructura, aplica las migraciones, siembra los datos y prepara el inventario en Redis.

Repartido en dos máquinas

- Computador A: `./iniciar.sh --rol datos`
- Computador B: `./iniciar.sh --rol servicios --datos <IP>`

Verificado: en la máquina de servicios arrancan **6** contenedores —los 4 servicios, el Gateway y el portal— y **ninguno** de la capa de datos.

Otras opciones del mismo guion

- `--replicas 2` — varias instancias de Entradas tras el balanceador
- `--registro develop` — despliega **sin código fuente**, desde las imágenes que publicó el pipeline
- `--datos-demo sql` — datos de ejemplo sin necesitar Node

Despliegue sin compilar nada

Una máquina con **solo Docker** levanta el sistema con datos usables: se bajan las siete imágenes ya construidas y se levantan. La diferencia es de minutos a segundos.

Integración continua — en cada pull request

- Guardia de **artefactos versionados** (nada de dist/ ni node_modules/)
- Chequeo de tipos, pruebas unitarias y compilación de los **4 servicios**
- Compilación de los **2 portales** Angular
- Análisis estático y pruebas de widget de la **app Flutter**
- Validación del `nginx.conf` y de los `docker-compose`
- **Pruebas de integración**: levanta el sistema con `./iniciar.sh` y corre las 20 suites

Entrega continua — en cada merge a develop o main

- Construye y publica **7 imágenes** multi-arquitectura (amd64 y arm64) en `ghcr.io`
- Tres etiquetas por imagen: la rama, el **commit exacto** (`sha-XXXXXXX`) y `latest`
- Un trabajo final **baja lo publicado y comprueba que arranca**

El guion de arranque, integrado al pipeline

El trabajo de integración usa el **mismo** `iniciar.sh` de la **demonstración**. Así el guion queda probado en cada PR, no solo el día de la entrega.

Cierre

Entregado y verificable

- SRS completo (IEEE 830) y SAD con las 15 secciones del formato
- **CU-006** (complejo) end-to-end, con sus 8 caminos alternos
- CU-001–005, CU-011–012, CU-017–018 y CU-027 end-to-end
- **5 atributos** de calidad con cifras reproducibles en una orden
- Sistema repartible en **2 computadores** desde un guion
- CI/CD con pruebas de integración y 7 imágenes publicadas

Lo que falta, sin adornos

- **CU-010 (evacuación)**, el segundo caso complejo, sin implementar
- Parqueaderos (CU-021–025) sin backend
- Personal propio (CU-007–009) sin backend
- Proveedores, pagos y reportes (CU-030–032) en placeholder
- La cobertura de integración está en **91–95 %**, no en 100 %

Preferimos decir qué falta a presentar como terminado algo que no lo está.

Gracias

Demostración en vivo a continuación